1. Cuentas, notificaciones y auditoría de Ahorro Futuro

1.1. Notas del bloque A

1.1.1. Interacciones del diagrama

1. **A1 y A2. Cliente móvil/web y terminal de ventanilla al Servicio de cuentas.** RPC síncrona mediante gRPC (gRPC-Web en el navegador) sobre HTTP/2 con TLS. Envían `ConsultarSaldo(idCuenta)` y reciben el saldo. El socio y el cajero esperan la respuesta, por lo que la comunicación es síncrona [1].

2. **A3. Servicio de cuentas al Core heredado.** Solicitud síncrona por la interfaz nativa del core, a través de su adaptador, con `ObtenerSaldo` y movimientos. Lleva un plazo de 1,5 s para que el socio reciba un error claro y no una espera indefinida [4]. El servicio de cuentas no guarda saldo propio.

3. **A4a y C6. Servicio de cuentas y Servicio de transferencias al exchange `operaciones`.** Publicación asíncrona por AMQP 0-9-1 en RabbitMQ, con tipo *topic*. Cuentas publica `ConsultaRealizada`. Transferencias publica `OperaciónConfirmada` después de que el core confirma. El Rol D publicará los eventos de retiro y reverso; esa flecha falta en el diagrama.

4. **A5. Cola `notificaciones.q` al Servicio de notificaciones.** Entrega asíncrona desde una cola durable. El servicio confirma el mensaje de forma manual solo tras procesarlo [7].

5. **A6. Cola `auditoria.q` al Servicio de auditoría.** Entrega asíncrona desde una cola *quorum*, replicada entre nodos del broker [8]. El servicio confirma después de guardar el evento. Los mensajes que no puede procesar pasan por un *dead letter exchange* a `auditoria.dlq` [9].

6. **A7. Servicio de notificaciones al proveedor push/SMS/correo.** Solicitud REST síncrona sobre HTTPS, con reintentos, fuera del flujo de la operación. Envía `aviso(idOperacion, canal)`.

7. **A8. Servicio de auditoría a la Base de auditoría.** Acceso a datos sobre TCP con el controlador de PostgreSQL. Inserta el evento y no actualiza ni borra registros.

1.1.2. Justificación del diseño

1. **R1 y R2. Consulta de saldo.** El saldo oficial solo existe en el core. Guardar uno en cuentas arriesgaría mostrar un saldo que no refleja un débito reciente. Se usa RPC síncrona porque alguien espera la respuesta. Se descartó la mensajería, que añade un salto, y REST con JSON sobre HTTP/1.1, que no ofrece plazo por llamada [4]. El plazo de 1,5 s es una decisión de diseño del equipo.

2. **R1 y R6. Notificaciones fuera del flujo crítico.** El aviso no es necesario para completar la operación. Si fuera síncrono, una caída del proveedor de SMS retrasaría

o rechazaría transferencias y retiros. Por eso el evento se publica después de que el core confirma y notificaciones lo consume cuando puede. Se usa publicación/suscripción porque el mismo evento alimenta a notificaciones y a auditoría sin que el publicador los conozca [10]. Se eligió RabbitMQ sobre Kafka porque el volumen del enunciado, hasta 300 transacciones por segundo, no exige un registro de eventos distribuido a gran escala; es un criterio del equipo, y Kafka sería la opción para reprocesar historia.

3. **R2 y F3. Entrega sin pérdidas y sin duplicados.** Los mensajes son persistentes, el publicador espera la confirmación del broker y los consumidores confirman tras procesar [7]. La entrega es al menos una vez, por lo que puede haber duplicados. Cada evento lleva `idOperacion` e `idEvento`, y los consumidores ignoran los repetidos [5]. Así el socio no recibe dos avisos por una misma operación.

4. **R5. Auditoría caída.** Los eventos esperan en `auditoria.q` y se procesan al recuperarse. Si el broker no responde, el servicio publicador guarda el evento en su bandeja de salida junto con el estado y reintenta después [6]. La cola y la bandeja tienen espacio finito, por lo que se dimensionan y se alerta por acumulación. Si ni el broker ni la bandeja pueden guardar el evento, la operación se rechaza.

5. **R6. Disponibilidad.** Notificaciones y auditoría quedan fuera de la ruta crítica, así que su caída no detiene transferencias ni retiros. El servicio de cuentas no guarda estado y puede replicarse.

1.1.3. Alternativas descartadas

- REST con JSON sobre HTTP/1.1 para consultar saldo: mensajes más pesados y sin plazo por llamada.
- Mensajería para consultar saldo: añade un salto mientras alguien espera.
- Notificación síncrona dentro de la operación: una caída del proveedor frenaría las operaciones.
- Kafka: el volumen del enunciado no lo justifica.
- Saldo guardado en el servicio de cuentas: podría no reflejar un débito reciente.

2. Transferencia interna de Ahorro Futuro

2.1. Notas del bloque B

2.1.1. Interacciones del diagrama

1. **B1. Cliente móvil/web compartido (A) al Servicio de transferencias.** El cliente envía por REST sobre HTTPS el `idOperacion`, las cuentas de origen y destino y el monto. Conserva el identificador al repetir el intento. El servicio devuelve el identificador y el estado. Confirma el éxito después de verificar el movimiento en el core.

2. **B2. Servicio de transferencias al Registro de operaciones.** Antes de llamar a antifraude, registra por SQL sobre TCP con TLS la clave (socio, `idOperacion`), los datos y el estado. Una restricción de unicidad y una inserción atómica permiten que una sola solicitud obtenga la clave. Si se repiten los datos, se devuelve el estado existente. Si cambian, se rechaza el uso de esa clave [5].

3. **B3. Servicio de transferencias a Antifraude (C).** RPC síncrona mediante gRPC sobre HTTP/2. Envía el `idOperacion` y los datos de riesgo. Recibe la decisión y su motivo. Sigue el plazo de 200 ms de C [4]. Ante un rechazo registra el motivo. Ante un error o vencimiento queda pendiente de antifraude. El movimiento en el core requiere aprobación.

4. **B4 y B5. Servicio de transferencias al Adaptador y al Core heredado.** Tras la aprobación, el servicio envía el identificador, ambas cuentas y el monto al adaptador por gRPC. El adaptador solicita por la interfaz nativa del core el débito y el crédito en una sola transacción. El core devuelve el resultado y la referencia oficial. Si se pierde la respuesta, se consulta o concilia esa referencia antes de reintentar.

5. **B6a. Resultado y evento en el Registro de operaciones.** El servicio guarda en una misma transacción SQL el estado y el evento pendiente en la bandeja de salida. Conserva `idEvento`, `idOperacion`, estado, fecha y referencia del core cuando exista. Si el core confirma y falla la escritura local, la conciliación recupera el resultado oficial y registra el evento con un identificador estable [6].

6. **B6b y B6c. Publicador de outbox y RabbitMQ compartido (A).** El publicador lee eventos pendientes por SQL y los envía por AMQP 0-9-1 al exchange operaciones, fuera del flujo principal. Espera confirmación del broker y reintenta conservando el `idEvento`. Auditoría deduplica por ese identificador y confirma después de procesar. Las notificaciones de éxito requieren que el core haya confirmado el movimiento [6].

2.1.2. Justificación del diseño

1. **R2 y F3. Consistencia y doble envío.** El registro distingue dos operaciones válidas de un doble clic. El core debe admitir una referencia única consultable para conciliar sin repetir el débito. Si sigue sin conocerse el resultado, se mantiene pendiente y se deriva a conciliación antes de considerar un reintento [5].

2. **R3 y F2. Antifraude fuera de servicio.** La transferencia queda pendiente de evaluación cuando se agota el plazo de 200 ms. Puede retomarse con el mismo identificador cuando antifraude esté disponible.

3. **R5. Auditoría y notificaciones.** La bandeja de salida cubre una caída del broker y la cola durable de A conserva los eventos cuando cae auditoría. El envío directo sin guardar el evento dejaría operaciones sin registro después de una interrupción [6].

4. **R1 y R6. Rendimiento y disponibilidad.** Se mediría si, bajo 300 transacciones por segundo, al menos el 95 % llega a estado final en menos de 2 segundos. Un estado pendiente queda fuera de las completadas. Se propone replicar el servicio y su registro

con conmutación ante fallos, y verificar el 99,95 % con pruebas de caída y recuperación del core, del registro y de antifraude.

3. Transferencia interbancaria y antifraude de Ahorro Futuro

3.1. Notas del bloque C

3.1.1. Interacciones del diagrama

1. Servicio de transferencias a Registro de operaciones. Consulta y escritura síncronas mediante un controlador SQL sobre TCP con TLS. El servicio registra la clave compuesta por socio e `idOperacion`, los datos de la solicitud y el estado de la operación. Una restricción de unicidad y una inserción atómica deben impedir que dos solicitudes reserven la misma clave. Si se repite una solicitud con los mismos datos, se devuelve el estado o resultado registrado. Si se reutiliza el identificador con datos diferentes, se propone rechazar la solicitud. Esta reserva corresponde al identificador de la operación, no a una reserva de fondos. El motor SQL y su controlador quedan por definir.

2. Servicio de transferencias a Servicio antifraude. RPC síncrona mediante gRPC sobre HTTP/2. El orquestador envía `idOperacion` y los datos necesarios para evaluar el riesgo. El servicio antifraude aplica las reglas configuradas y produce un dictamen. La elección de RPC permite expresar la interacción como una llamada remota con parámetros y resultados definidos, aunque deben contemplarse los fallos propios de la comunicación entre procesos [1]. El plazo de 200 ms es una propuesta del diseño y debe validarse mediante pruebas.

3. Servicio antifraude a Servicio de transferencias. Respuesta de la misma llamada RPC con decisión, motivo e `idOperacion`. Si el dictamen es aprobado, el orquestador guarda el resultado y continúa con la solicitud de reserva de fondos. Si es rechazado, registra el motivo y no envía la transferencia al banco. Si ocurre un error o vence el plazo, registra la operación como pendiente de antifraude y suspende el envío. El vencimiento puede ser detectado por el cliente sin recibir una respuesta válida. La ausencia de respuesta no se interpreta como un rechazo por riesgo.

4. Servicio de transferencias a Núcleo bancario heredado. Petición y respuesta síncronas mediante la interfaz nativa del core. Después de la aprobación antifraude, el orquestador envía los datos necesarios para consultar la cuenta y solicitar la reserva de fondos. El core devuelve el resultado, `idOperacion` y `referenciaCore`. Solo se continúa hacia la pasarela cuando la reserva ha sido aceptada. Si el core rechaza la solicitud o no puede determinarse el resultado, se suspende el envío. El protocolo y el middleware deben precisarse según la interfaz disponible.

5. Servicio de transferencias a Pasarela interbancaria. Comunicación síncrona mediante sockets TCP con TLS y una API de sockets nativa. El servicio envía una trama con longitud, `idOperacion`, cuentas de origen y destino y monto. El envío requiere

aprobación antifraude y reserva de fondos aceptada. El contrato de comunicación debe definir el formato de los campos, los límites de longitud y los códigos de respuesta.

6. Pasarela interbancaria a Red interbancaria o banco destinatario. Solicitud y respuesta síncronas mediante sockets TCP con TLS. La pasarela adapta la solicitud al protocolo acordado con el sistema externo y recibe un acuse de recibo, una aceptación o un rechazo, según el contrato. La respuesta incluye una referencia cuando corresponda. Debe distinguirse entre recepción técnica, aceptación para procesamiento y confirmación definitiva. Un acuse de recepción no se considera suficiente para registrar éxito definitivo, salvo que el contrato establezca expresamente ese significado.

7. Pasarela interbancaria a Servicio de transferencias. Retorno del resultado sobre la conexión de la interacción 5. El orquestador registra la operación como confirmada, rechazada por el banco o por conciliar. Si se pierde la respuesta después del envío, no se presupone que la operación fracasó. Se consulta o concilia el estado mediante las referencias disponibles antes de reintentar. Una llamada remota puede dejar incertidumbre sobre la ejecución cuando falla la comunicación, por lo que el tratamiento de fallos forma parte del diseño [1].

8. Servicio de transferencias a Registro de operaciones. Escritura síncrona mediante SQL sobre TCP con TLS. El servicio guarda el resultado y el evento pendiente en un outbox dentro de la misma transacción local. Se conservan `idOperacion`, estado, motivo e `idEvento`. El objetivo es dejar registrado el trabajo de publicación junto con el cambio de estado. Esta transacción no incluye automáticamente al core ni al banco destinatario. La separación entre transacciones locales y comunicación entre sistemas requiere conservar estado y coordinar el procesamiento posterior de los mensajes [3].

9. Publicador de outbox a Registro de operaciones y RabbitMQ. El publicador consulta los eventos pendientes mediante acceso SQL síncrono y los publica mediante AMQP y RabbitMQ. La publicación es asíncrona respecto del flujo principal de la transferencia. Se propone utilizar mensajes persistentes, colas durables y confirmaciones del publicador para seguir la aceptación de los mensajes por el broker. Si no se obtiene confirmación, se gestiona el reintento sin asumir que la primera publicación necesariamente fracasó. El tratamiento de mensajes repetidos forma parte de la coordinación entre sistemas independientes [3].

10. RabbitMQ a Auditoría y notificaciones. Entrega asíncrona mediante AMQP. El consumidor deduplica por `idEvento`, procesa el resultado y envía el ACK después de completar el procesamiento previsto. El registro de deduplicación debe coordinarse con los efectos del consumidor para evitar que una repetición produzca efectos indebidos. Este diseño no presupone una garantía automática de procesamiento exactamente una vez [3]. Las notificaciones de éxito definitivo se generan únicamente cuando el resultado bancario permite establecer dicho estado.

3.1.2. Justificación del diseño

1. Latencia y respuesta inmediata. Antifraude, reserva de fondos e intercambio con la pasarela se resuelven de forma síncrona porque sus resultados condicionan la continuación

del flujo. El plazo de 200 ms limita la espera propuesta para antifraude, pero no representa la duración total de la transferencia. La latencia completa debe medirse incluyendo persistencia, evaluación de riesgo, core, pasarela y respuesta bancaria.

2. Elección de RPC y sockets. RPC se utiliza para representar la evaluación antifraude como una operación remota con parámetros y resultados definidos [1]. gRPC sobre HTTP/2 es la tecnología propuesta para implementarla. Los sockets TCP con TLS se reservan para el intercambio explícito de tramas con la pasarela y la red interbancaria. La elección permite implementar ambos mecanismos, aunque no demuestra que uno tenga mejor rendimiento. Su comparación requiere pruebas bajo condiciones equivalentes.

3. Consistencia e identificación de solicitudes. El registro persistente conserva la relación entre el identificador, los datos de la solicitud y su estado. La restricción de unicidad evita reservar dos veces la misma clave, pero no basta por sí sola para impedir ejecuciones externas duplicadas. El orquestador también debe controlar las transiciones de estado y utilizar mecanismos de identificación o consulta admitidos por el core y la red. La comunicación entre entidades con transacciones independientes requiere tratar explícitamente las repeticiones y la evolución del estado [3].

4. Coordinación con el core. La reserva de fondos y el envío interbancario no se consideran una única transacción SQL. Se propone coordinar sus resultados y definir las acciones posteriores: confirmar el movimiento, liberar la reserva o mantener la operación pendiente de conciliación. El modelo de sagas proporciona un fundamento para organizar secuencias de transacciones y acciones compensatorias cuando no se utiliza una sola transacción global [2]. Esto no significa que una transferencia ya confirmada por el banco pueda deshacerse mediante una simple reversión local.

5. Tolerancia a fallos. El outbox conserva eventos pendientes para permitir su recuperación después de una interrupción. Los reintentos deben acompañarse de identificación de mensajes, deduplicación y control de estado. Esta separación permite coordinar transacciones locales y mensajes sin suponer que todos los participantes comparten una operación atómica [3].

6. Volumen de mensajes. Auditoría y notificaciones quedan fuera de la espera principal de la transferencia. Deben medirse la capacidad del publicador, el ritmo de consumo y la acumulación de eventos. El diagrama no demuestra una capacidad concreta de transacciones por segundo. Cualquier objetivo de rendimiento debe verificarse mediante pruebas de carga y recuperación después de fallos.

7. Alternativas descartadas. No se continúa hacia la pasarela sin aprobación antifraude ni reserva aceptada. No se confirma el éxito a partir de un acuse que solo indique recepción. Tampoco se reintenta una operación de resultado incierto sin consultar o conciliar su estado. Para auditoría se propone publicación con registro persistente en lugar de depender únicamente de un envío sin recuperación. La persistencia del estado y el tratamiento de mensajes repetidos son necesarios cuando la comunicación conecta transacciones independientes [3].

Referencias

[1] A. D. Birrell y B. J. Nelson, "Implementing remote procedure calls," *ACM Transactions on Computer Systems*, vol. 2, núm. 1, pp. 39–59, 1984. DOI: 10.1145/2080.357392.

[2] H. Garcia-Molina y K. Salem, "Sagas," en *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, pp. 249–259, 1987. DOI: 10.1145/38713.38742.

[3] P. Helland, "Life beyond distributed transactions," *Communications of the ACM*, vol. 60, núm. 2, pp. 46–54, 2017. DOI: 10.1145/3009826.

[4] gRPC Authors, "Deadlines", *gRPC Documentation*. `https://grpc.io/docs/guides/deadlines/`.

[5] M. Featonby, "Making retries safe with idempotent APIs", *The Amazon Builders' Library*. `https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/`.

[6] Amazon Web Services, "Transactional outbox pattern", *AWS Prescriptive Guidance*. `https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html`.

[7] RabbitMQ, "Consumer Acknowledgements and Publisher Confirms", *RabbitMQ Documentation*. `https://www.rabbitmq.com/docs/confirms`.

[8] RabbitMQ, "Quorum Queues", *RabbitMQ Documentation*. `https://www.rabbitmq.com/docs/quorum-queues`.

[9] RabbitMQ, "Dead Letter Exchanges", *RabbitMQ Documentation*. `https://www.rabbitmq.com/docs/dlx`.

[10] G. Hohpe y B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston, MA, EE. UU.: Addison-Wesley, 2003, ISBN 978-0-321-20068-6.